CHROMA LAB



Ask your documents. Get grounded answers.

A Retrieval-Augmented Generation (RAG) workspace powered by FastAPI, ChromaDB and Groq.

PROJECT OVERVIEW

Prepared for academic presentation • AI & Data Science

The problem: knowledge is scattered

Important information is often buried inside files, making it slow to find and difficult to verify.

01 Search friction

Users may need to open several documents and scan them manually to locate one useful detail.

02 Context switching

Switching between files interrupts workflow and makes comparison harder.

03 Trust & traceability

A useful answer should be connected to the source material—not presented as an unsupported guess.

Design goal: turn a collection of documents into a searchable, conversational knowledge space.

What is Chroma Lab?

A compact web workspace for uploading knowledge files and asking questions about their contents.

DOCUMENT INGESTION

Upload supported files and prepare their text for indexing.

SEMANTIC RETRIEVAL

Retrieve relevant text chunks using vector similarity search.

GROUNDED ANSWERS

Pass retrieved context to the language model to form a response.

SOURCE AWARENESS

Show source information so users can trace an answer to indexed material.

Core idea: Retrieval-Augmented Generation (RAG) combines retrieval with generation.

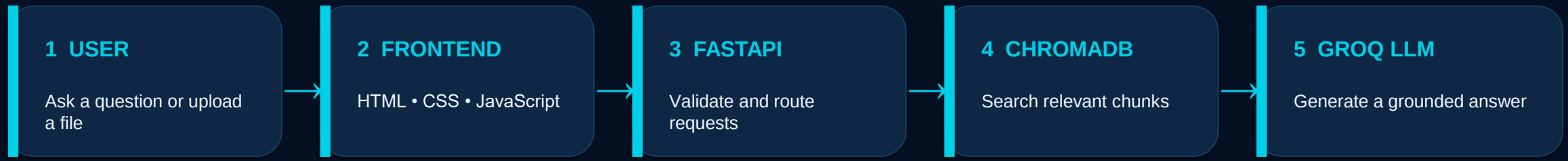
Objectives & scope

The project focuses on making document-based question answering practical and understandable.

- Provide a clean, dark-themed interface for document-based work.
- Accept knowledge files and index their content into a vector store.
- Convert a natural-language question into a retrieval request.
- Use retrieved document context to generate a relevant answer.
- Expose source details and basic document/index status.
- Keep the frontend connected to a FastAPI backend through defined endpoints.

System architecture

A modular pipeline separates the user interface, API orchestration, retrieval and answer generation.



INGESTION PATH

File → text extraction → chunking → embeddings → vector store

QUESTION PATH

Question → retrieval → context prompt → generated answer + source

Conceptual architecture; exact implementation details depend on the configured loaders and service classes.

Technology stack

Each component has a focused responsibility in the end-to-end workflow.

HTML

Page structure and workspace regions

CSS

Dark navy / cyan theme, spacing and responsive layout

JavaScript

Frontend interactions and API requests

Python + FastAPI

Backend routes and request handling

ChromaDB

Persistent vector database for document chunks

Sentence Transformers

Text embeddings for semantic similarity

Groq API

Language-model response generation

RAG in plain English

The answer is generated with retrieved evidence from the user's indexed documents.

1

Retrieve

Find the document chunks most relevant to the question.

2

Augment

Add those chunks as context for the language model.

3

Generate

Ask the model to formulate a concise answer from the context.

4

Attribute

Return source information alongside the answer where available.

Why it matters: retrieval helps connect generated answers to the documents selected for the current question.

User journey

From a local knowledge file to a source-grounded response.

01	**Open workspace**	Frontend loads the document list and backend status.
02	**Add documents**	Upload a supported file and process its text.
03	**Index content**	Split text into chunks and store embeddings/metadata.
04	**Ask naturally**	Enter a question in the chat composer.
05	**Review response**	Read the answer and inspect the cited source details.

Interface design

The UI is designed as a knowledge workspace rather than a generic chat page.

LEFT SIDEBAR

Navigation, backend status, knowledge statistics and upload controls.

CHAT WORKSPACE

Welcome panel, suggested prompts, conversation history and composer.

DOCUMENT PANEL

Indexed document list with file names and chunk counts.

Visual principles

- Dark navy surfaces with cyan accents for a focused AI-tool aesthetic.
- Compact spacing to give conversation content more usable room.
- Clear visual hierarchy between navigation, workspace and indexed sources.
- Responsive styling for narrower screens, subject to browser/device testing.

Backend API overview

The frontend communicates with the FastAPI application through HTTP endpoints.

GET /health	Check whether the backend responds.
GET /documents	Return indexed document names and chunk counts.
POST /upload	Accept and process document uploads.
POST /reset	Reset the current vector collection.
POST /rag/query	Retrieve context and generate a response.

API names shown here reflect the current project structure described during development.

Testing & observed results

Core local workflow has been tested during development.

BACKEND HEALTH

FastAPI starts locally and exposes the interactive API documentation.

DOCUMENT LIST

The API returned an indexed file and its chunk count.

RAG RESPONSE

A test question about ChromaDB returned an answer grounded in indexed notes.

Example local test

Question: “What is ChromaDB?”

Observed response: “ChromaDB is a vector database used to store and search embeddings.”

These are local development checks, not a claim of public production deployment or formal load testing.

Benefits & practical use cases

Chroma Lab is useful when answers should be based on a specific set of documents.

STUDY & RESEARCH

Ask questions about notes, guides and learning resources.

TEAM KNOWLEDGE

Explore internal guides, FAQs and project documentation.

DOCUMENT REVIEW

Summarize or locate relevant passages without scanning every file.

Potential value

- Less manual searching across multiple documents.
- Natural-language interaction with an indexed knowledge set.
- Source-oriented answers that are easier to verify.
- Modular architecture that can be extended with authentication, monitoring and deployment.

Limitations & future roadmap

A transparent roadmap separates current local functionality from future production features.

CURRENT CONSIDERATIONS

- Local API currently requires the backend to be running.
- Answer quality depends on document text, chunking, retrieval and model output.
- Supported file formats depend on configured loaders.
- Production hosting and cloud persistence still need verification.

ROADMAP

- Push clean code and documentation to GitHub.
- Deploy frontend and backend with correct API URL and CORS settings.
- Store secrets in deployment environment variables.
- Add upload limits, clearer errors and stronger validation.
- Add authentication and monitoring if required.

Deployment principle: never expose the Groq API key in frontend code or commit secrets to GitHub.



CHROMA LAB

Make your documents easier to ask.

Thank you • Questions & Discussion